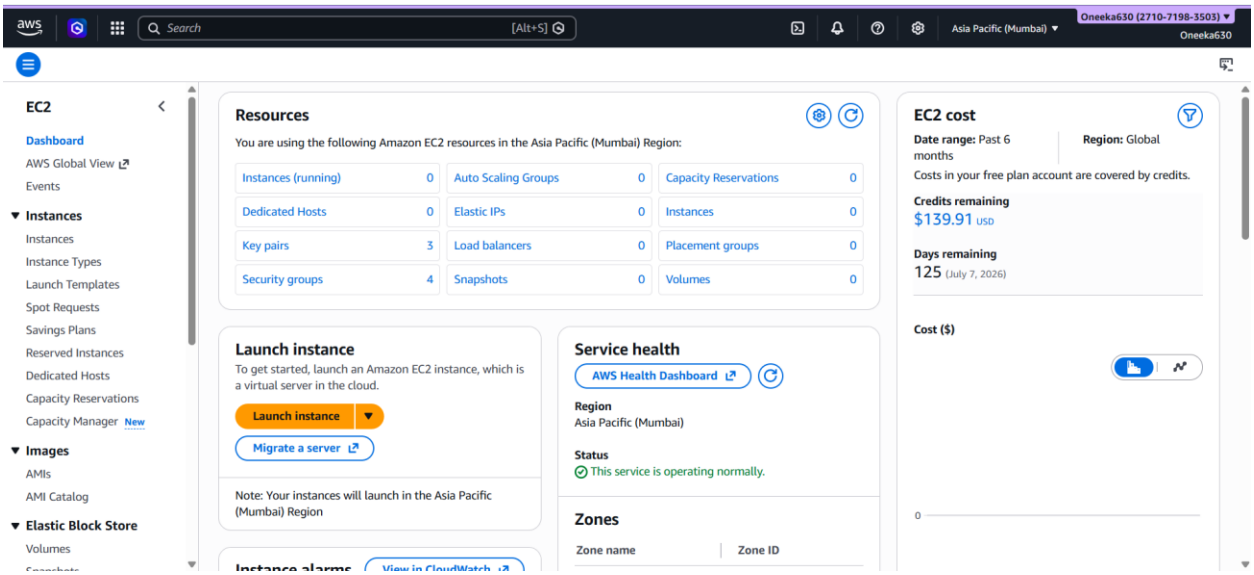
NAME :ONEEKA KUNDU **ROLL:129** **EXAMROLL:11600223145**

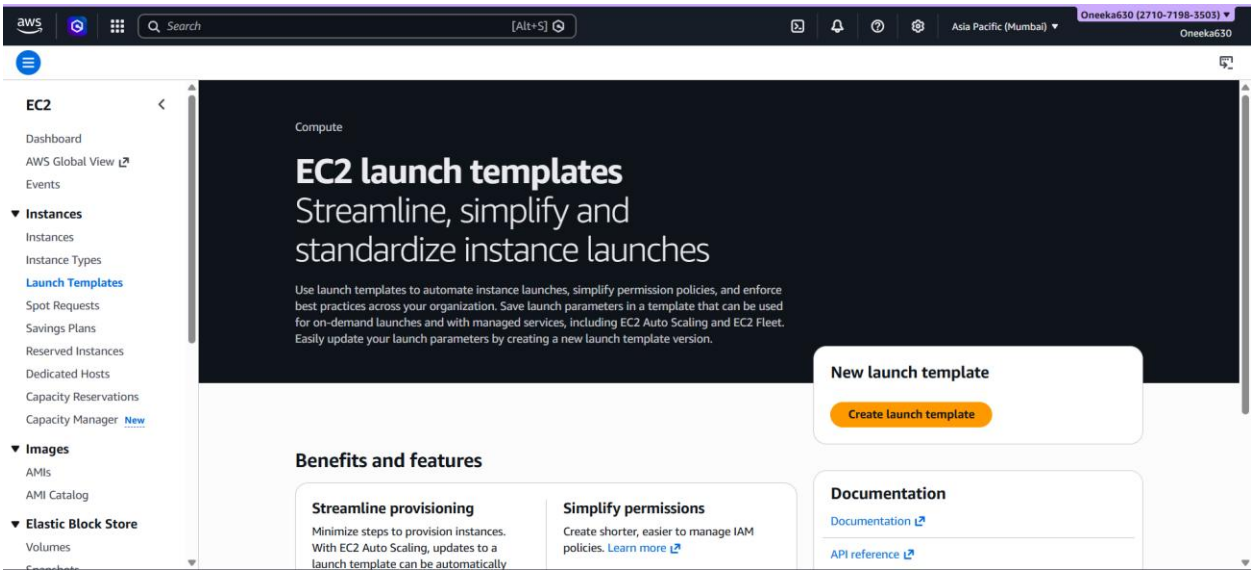
ASSIGNMENT: 11

PROBLEM STATEMENT: Build scaling plans in aws that balance the load on different EC2 instances.

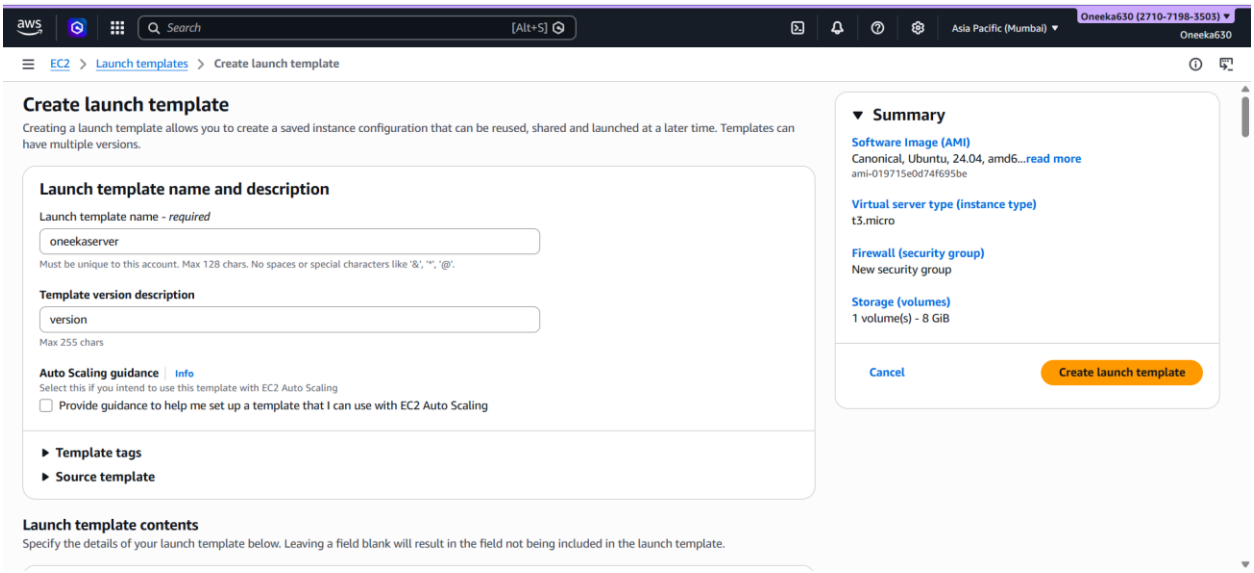
Step1: Sign up for an AWS account, search for '**EC2**' then click on it.



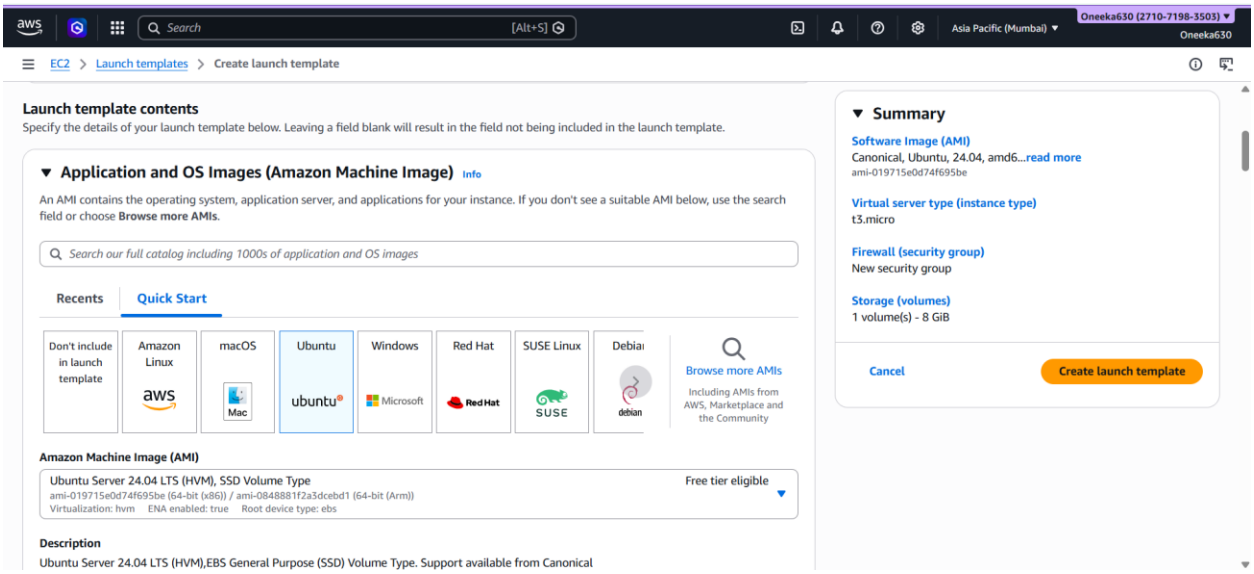
Step2: Click on '**Launch Templates'** & then '**Create launch template'**.



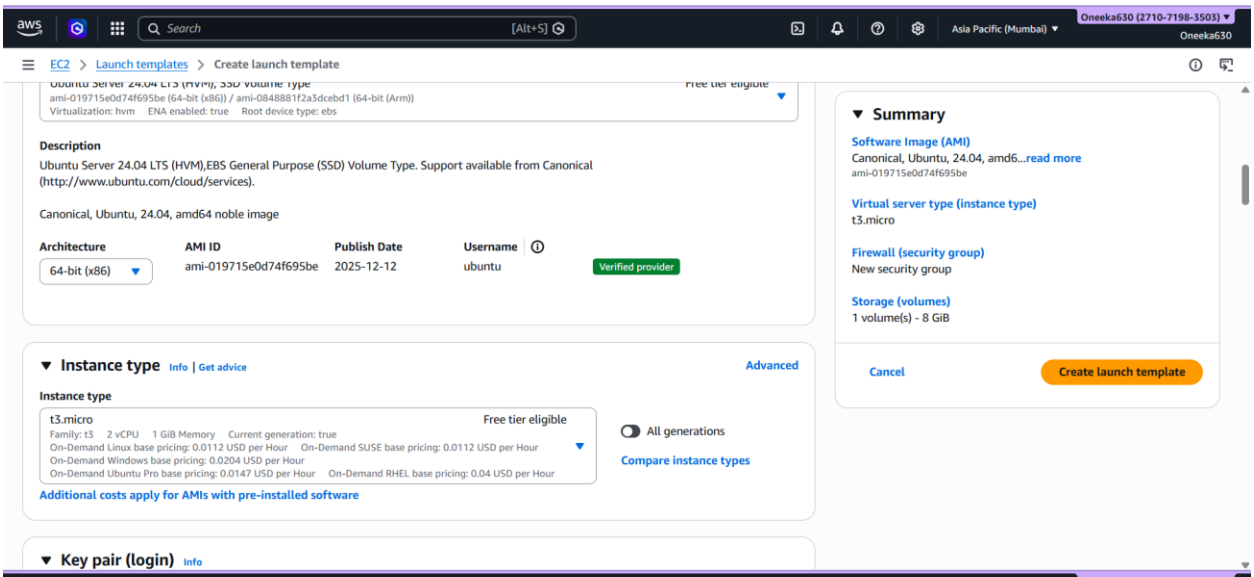
Step 3. In '**Launch templet name and description'** give a name and description to your launch templet and select the check box for '**Provide guidance to help me set up a template that I can use with EC2 Auto Scaling**'.

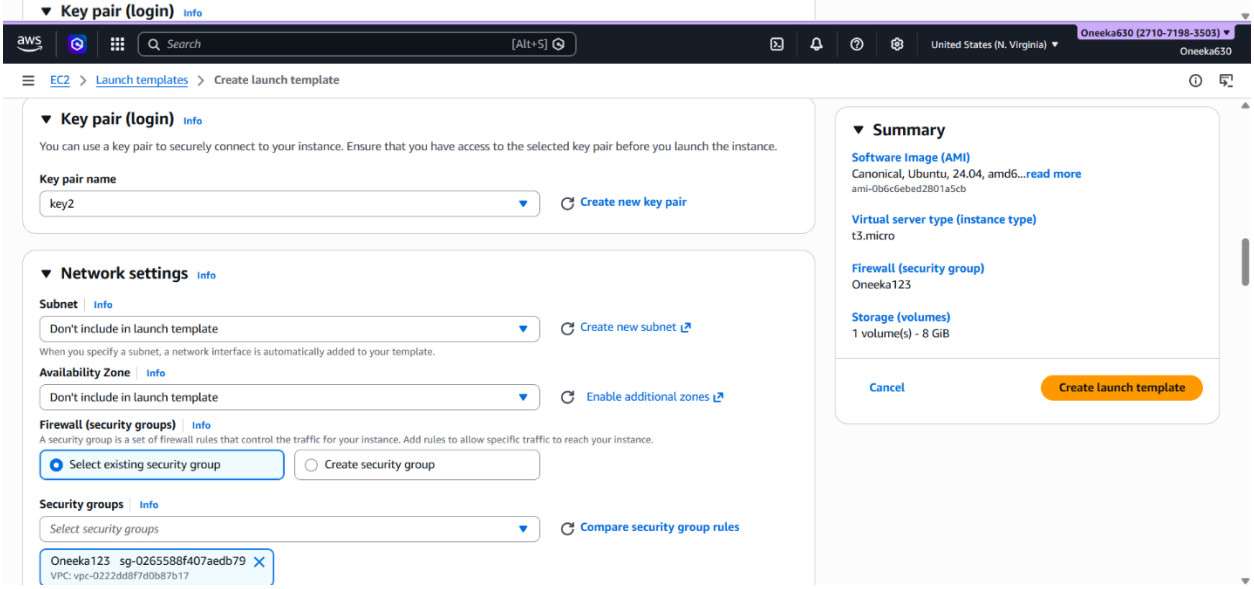


Step 4. For OS Image select '**ubuntu**'.

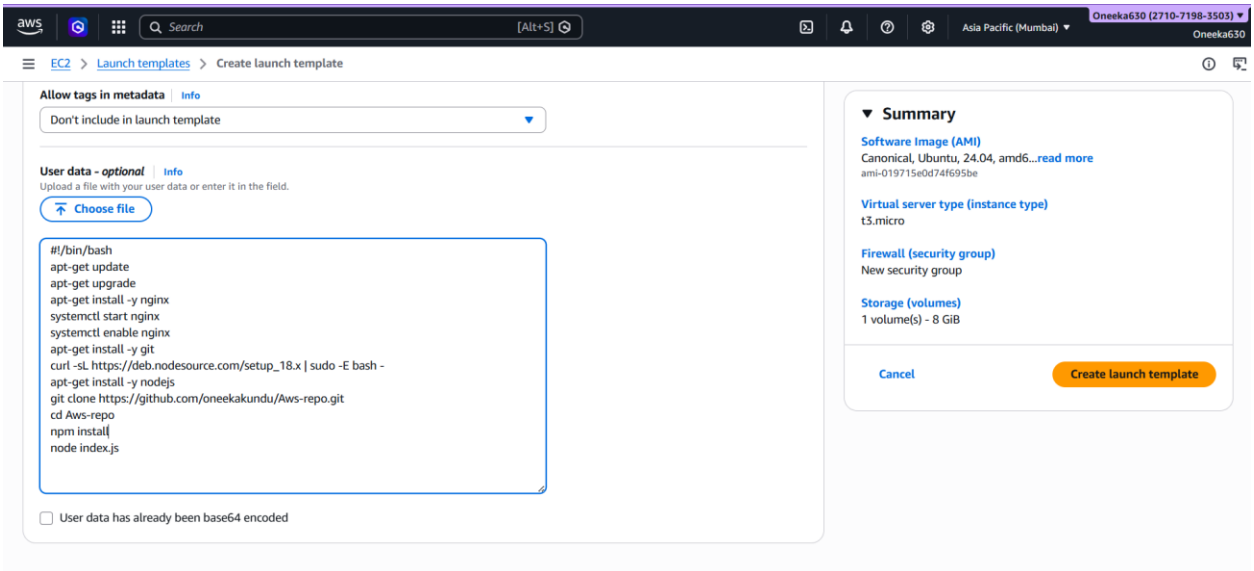


Step 5. For instance type, select '**t3.micro**' or any other free tire eligible instance type. For key pair, select an existing key pair or create a new key pair. For security group select an existing security group **where port 4000** is open for incoming traffic.

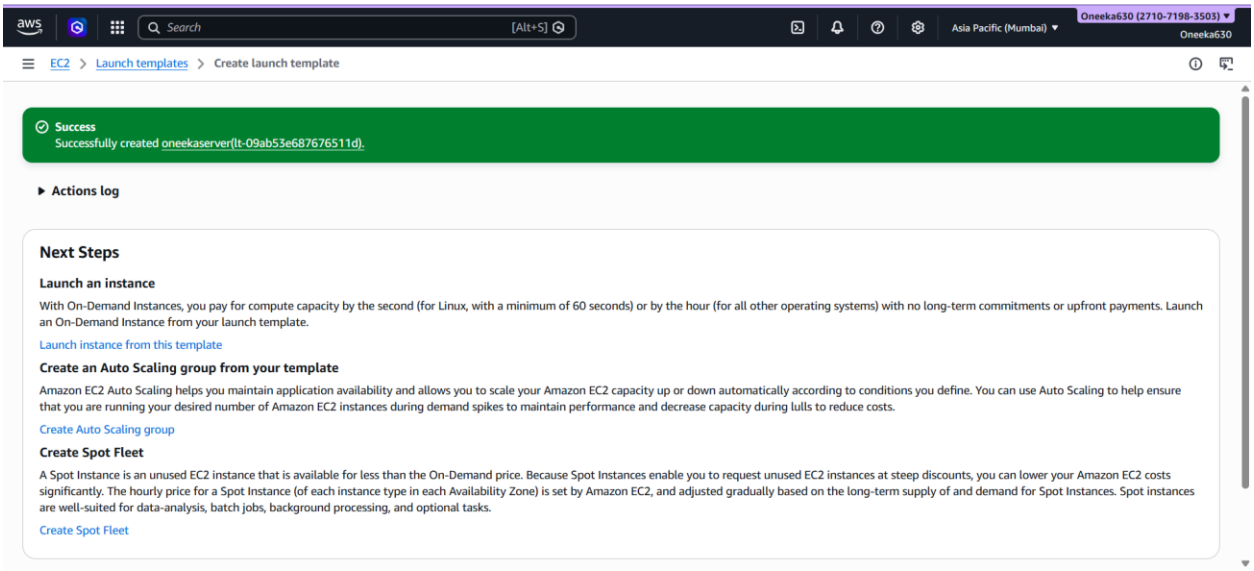




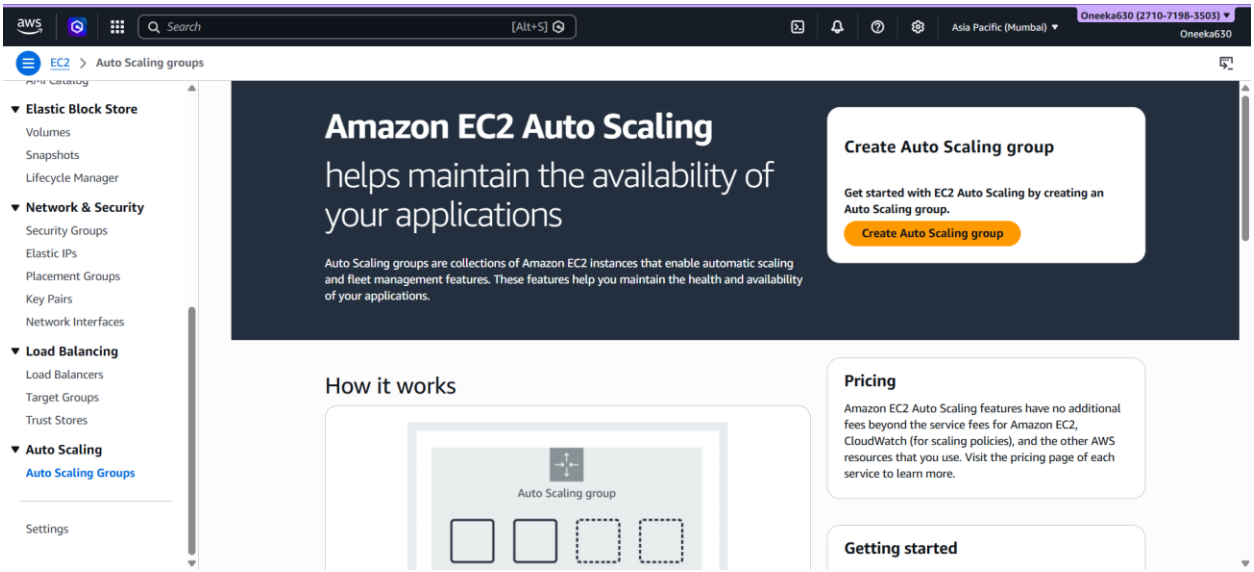
Step 6. **In Advanced details** scroll down to the bottom and add the text to the user data. Then click on **Create launch template**.



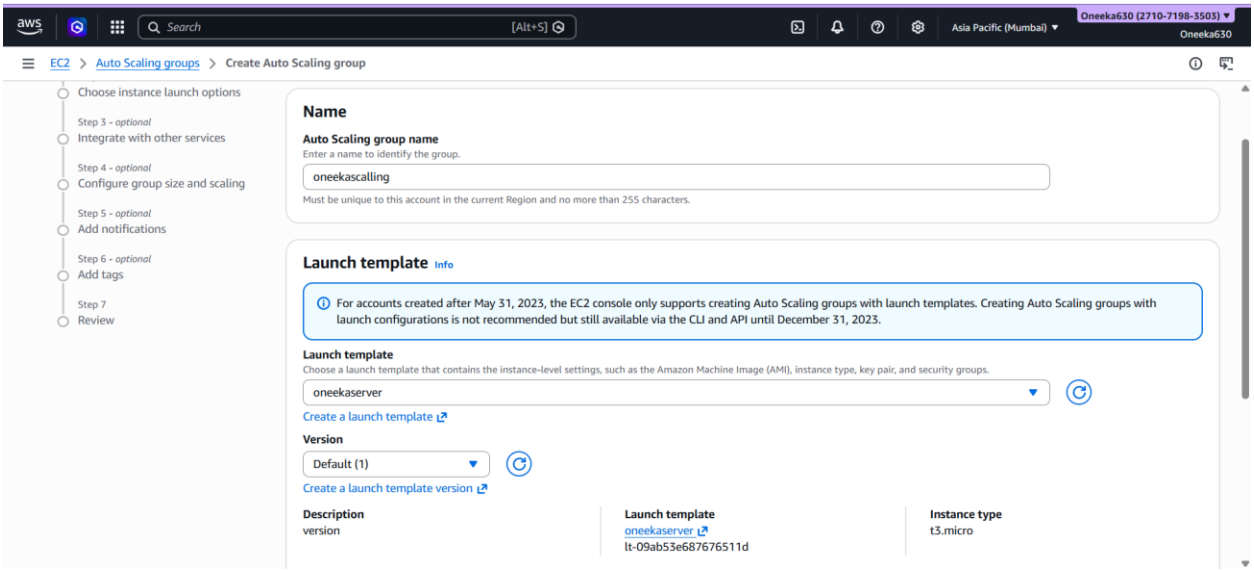
Step 7. Launch Templates created **successfully.**



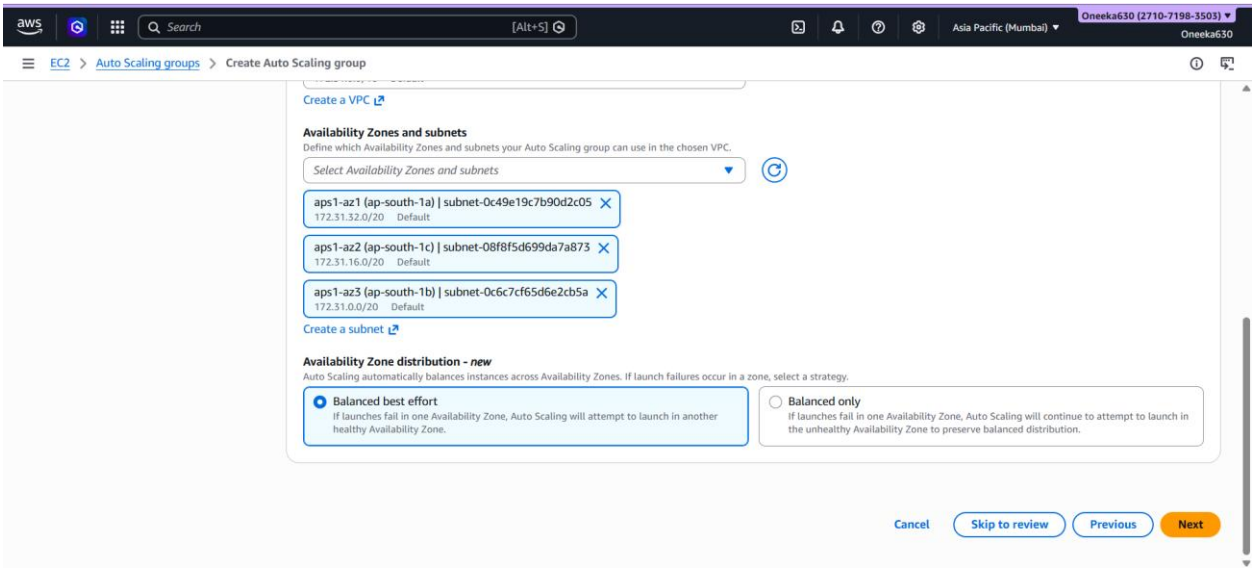
Step 8. Scroll down and click on **View launch templates** -> Now go to '**Create Auto Scaling Groups**'.



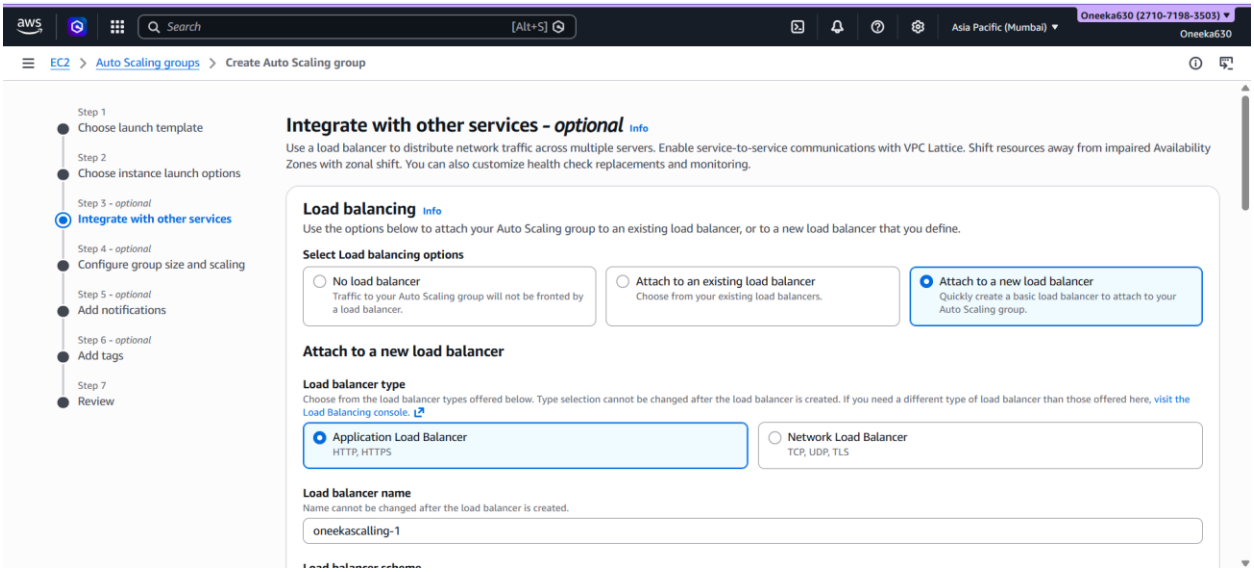
Step 9. Add Details for the Auto Scaling Group. In Step 1, Choose launch templates- Give a name to the Auto Scaling Group and choose the Launch template you just created as the launch template. Click on Next.



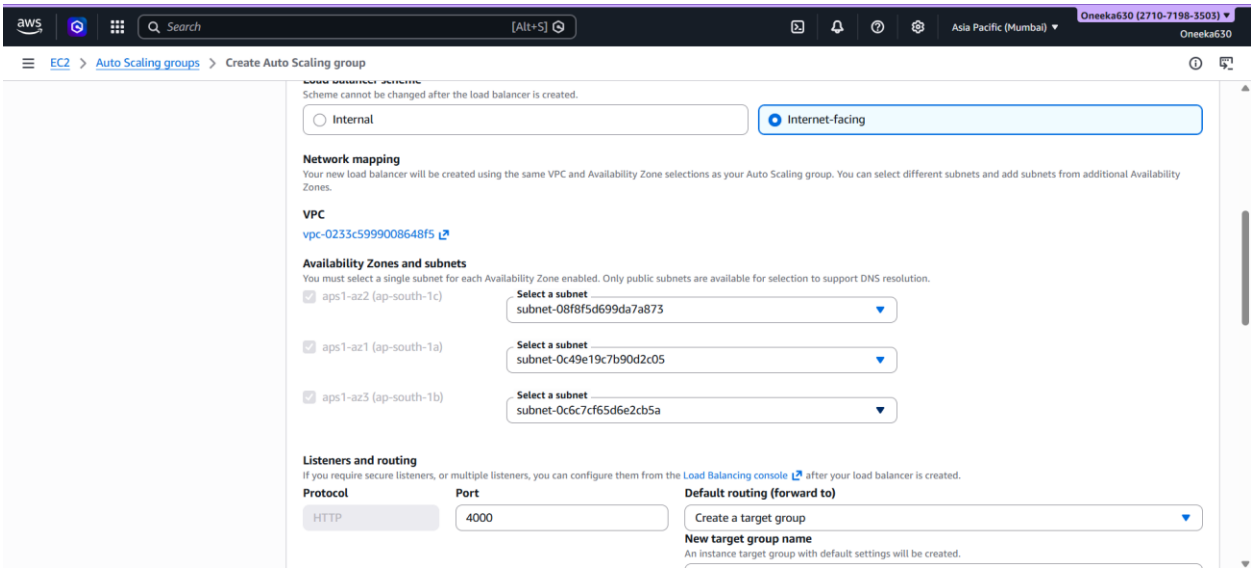
Step 10: In Step 2, Choose instance launch options- Choose all the '**Availability Zones**' available. Click on Next.

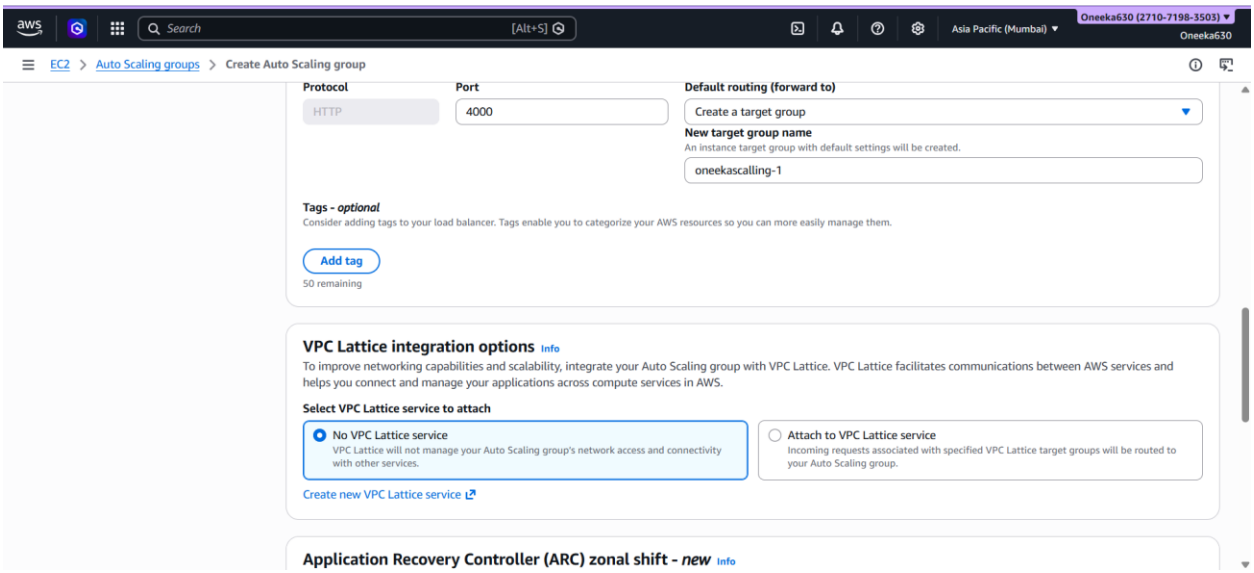


Step 11: In Step 3, Configure advanced options- For Load balancing select '**Attach to a new load balancer**'.

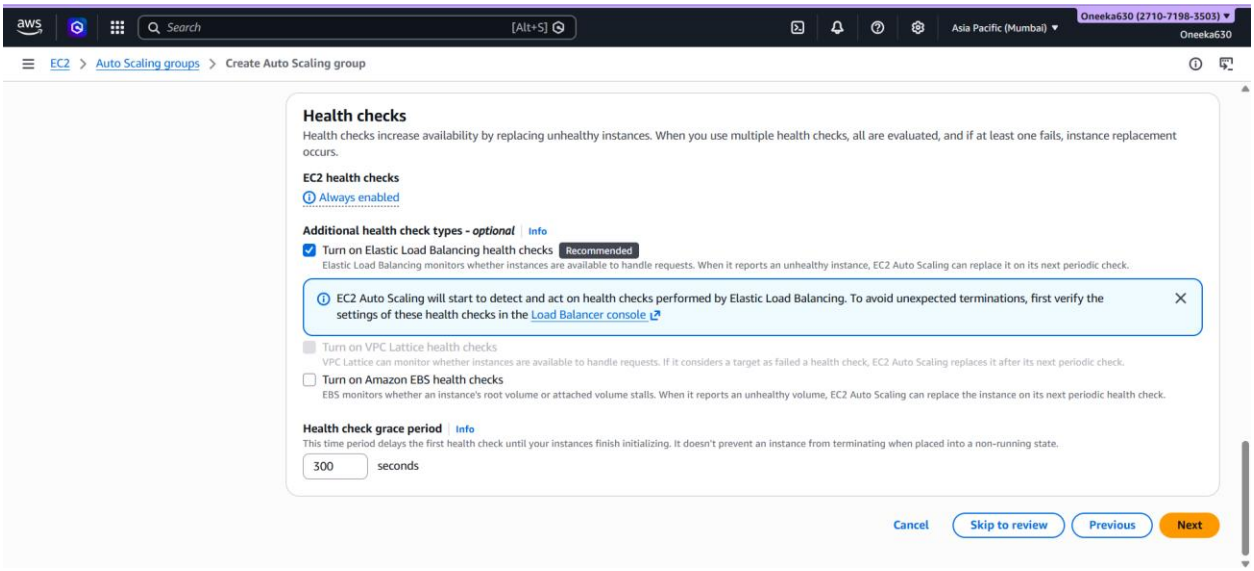


Step 12: For new load balancer configuration select '**Application Load Balancer**', '**Internet-facing**' and make HTTP port from 80 to **4000** while creating a target group.

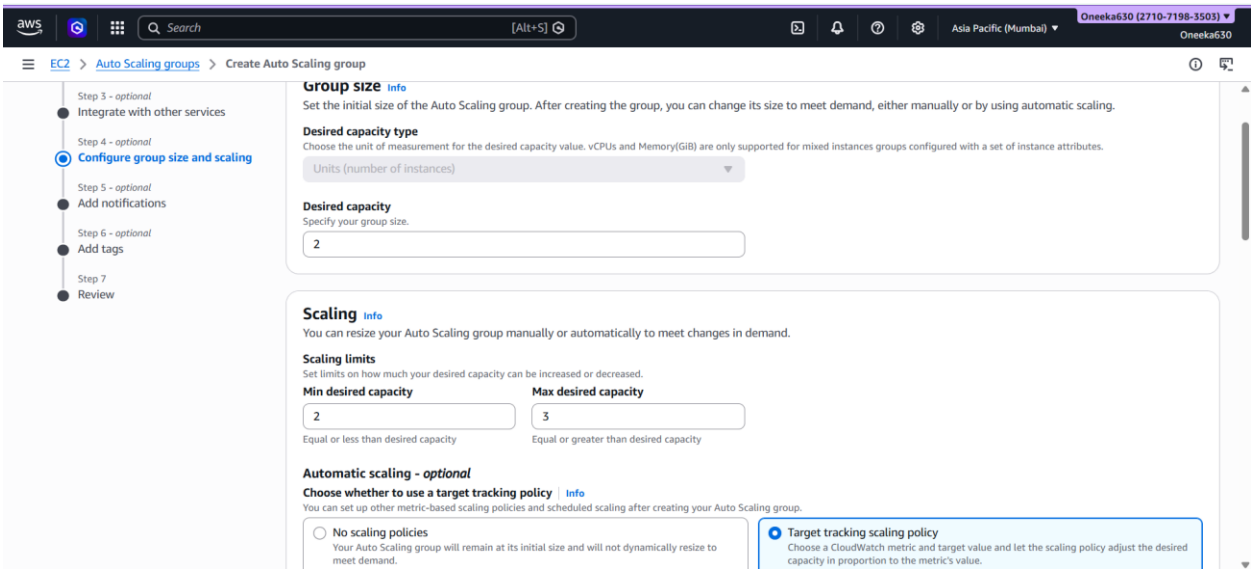




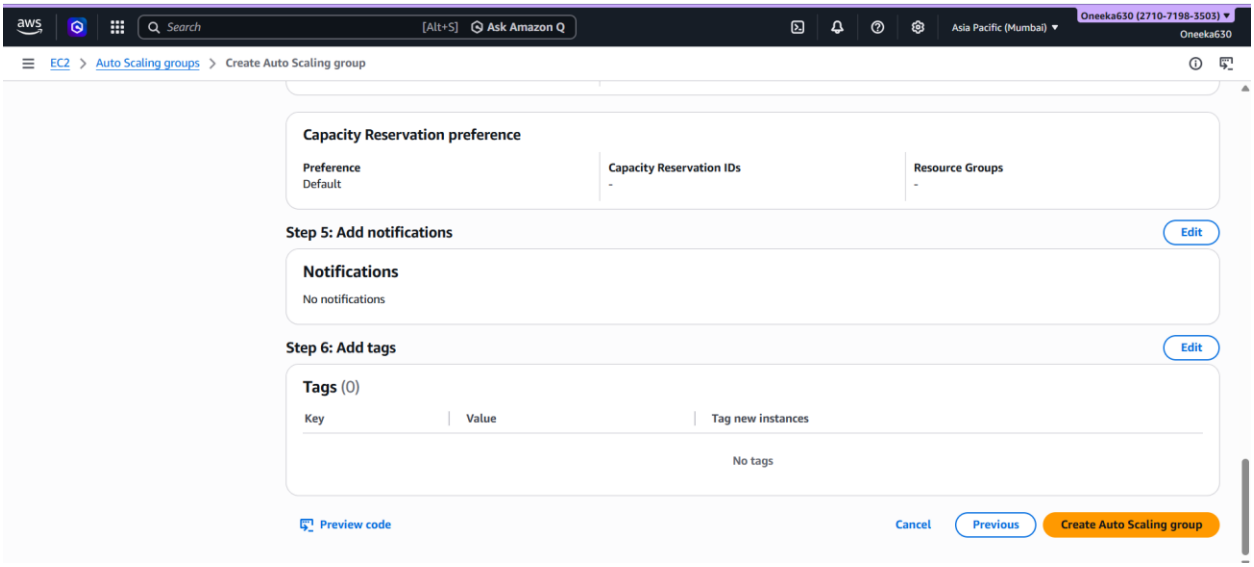
Step 13. Health checks tick the check box for '**Turn on Elastic Load Balancing health checks'** . Click on Next.



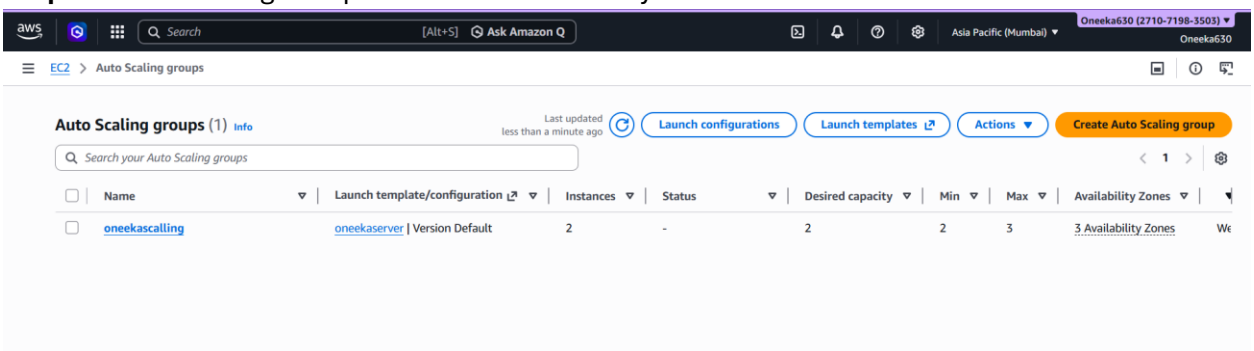
Step 14. In Step 4, Configure group size and scaling- In Group size set Desired capacity as 2. In Scaling set Min desired capacity as 2 and Max desired capacity as 3. In Automatic scaling choose target scaling policy .Keeping everything else as default, Click on Next.



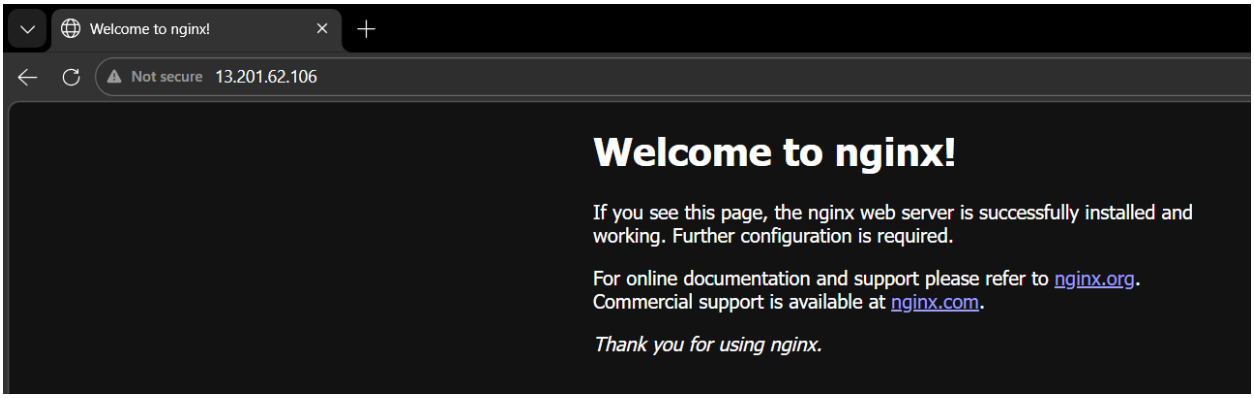
Step 15: Skip through Step 5 and Step 6 by clicking Next. Finally In Step 7 Click on Create Auto Scaling Group.



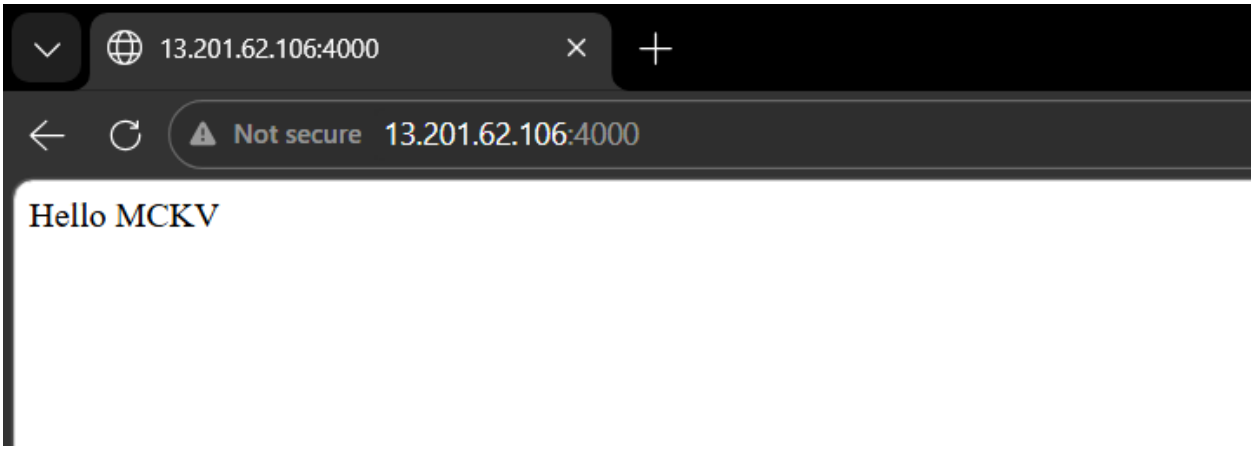
Step 16: Auto Scaling Group is created successfully.



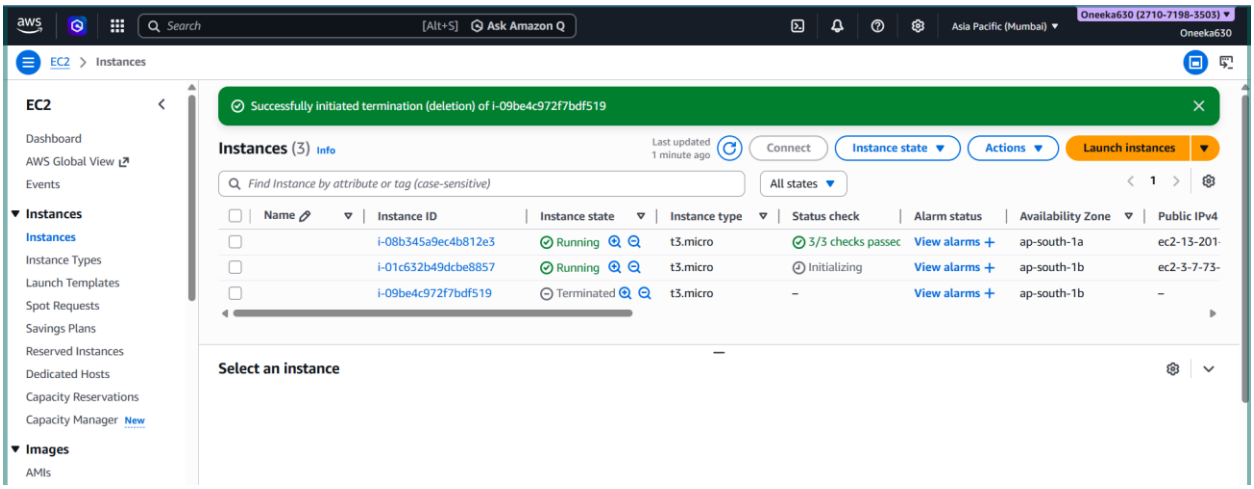
Step 17: Now in instances we can see two instances have been created->copy IPv4 of any instance and paste it



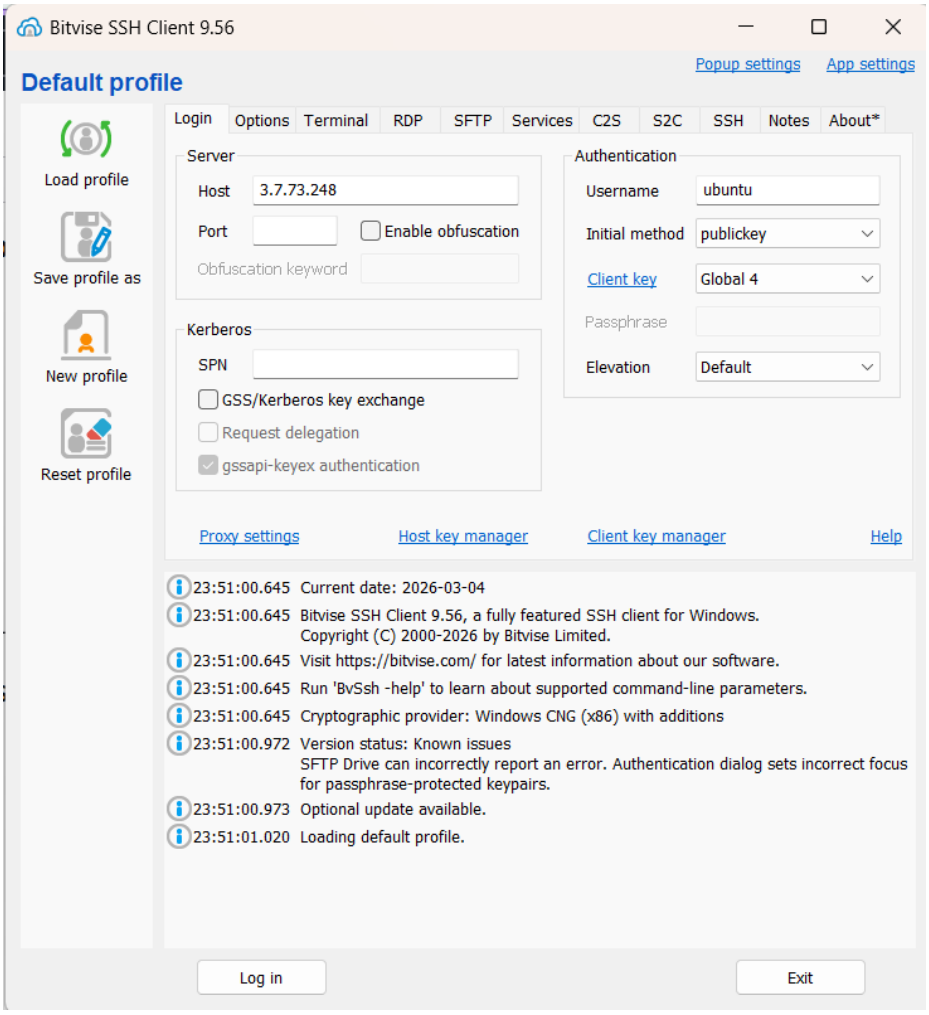
Step 18: Now paste the public ipv4 of any one instance and then add : 4000 to it.



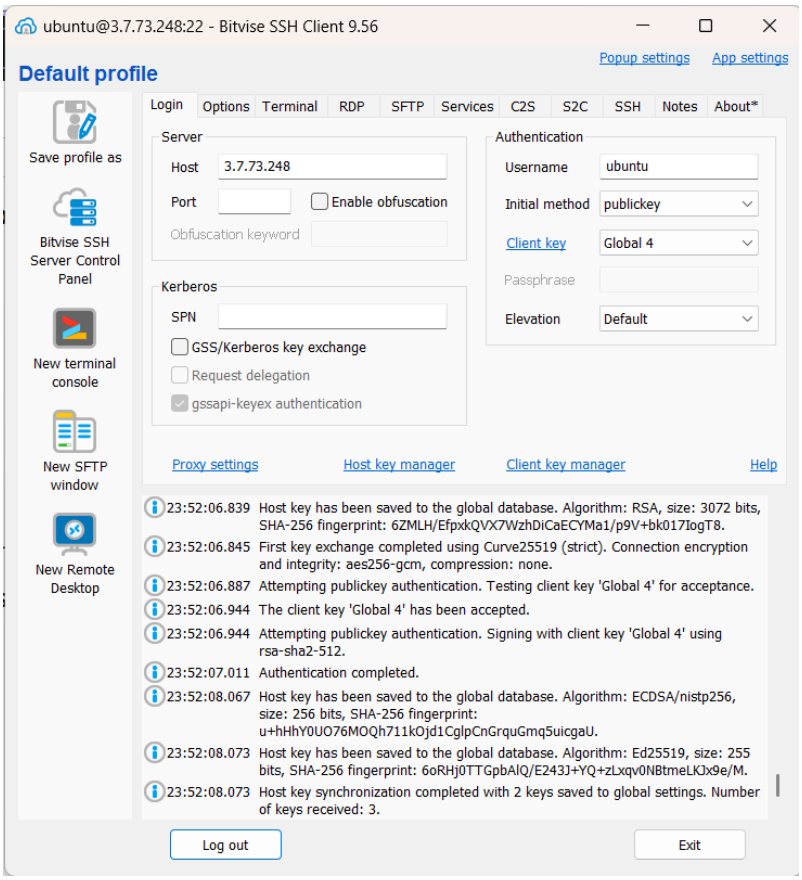
Step19: Now delete one instance and we will see that one new instance is created automatically.



Step 20: In Bitvise SSH Client, paste the **'Public IPV4 address'** and click on **'Client key Manager'**. 20. In Bitvise SSH Client, paste the **'Public IPV4 address'** and click on '**Client key Manager'**. 22. In '**Bitvise SSH Client'**, under '**Default profile'** give the **'Username'** as '**ubuntu**' then select '**Global4**' from '**Client Key Manager'** then click on **'Log in'** & **'Accept and save'**.



Step 21: Click on '**New terminal console'**, to access the terminal of your instance.



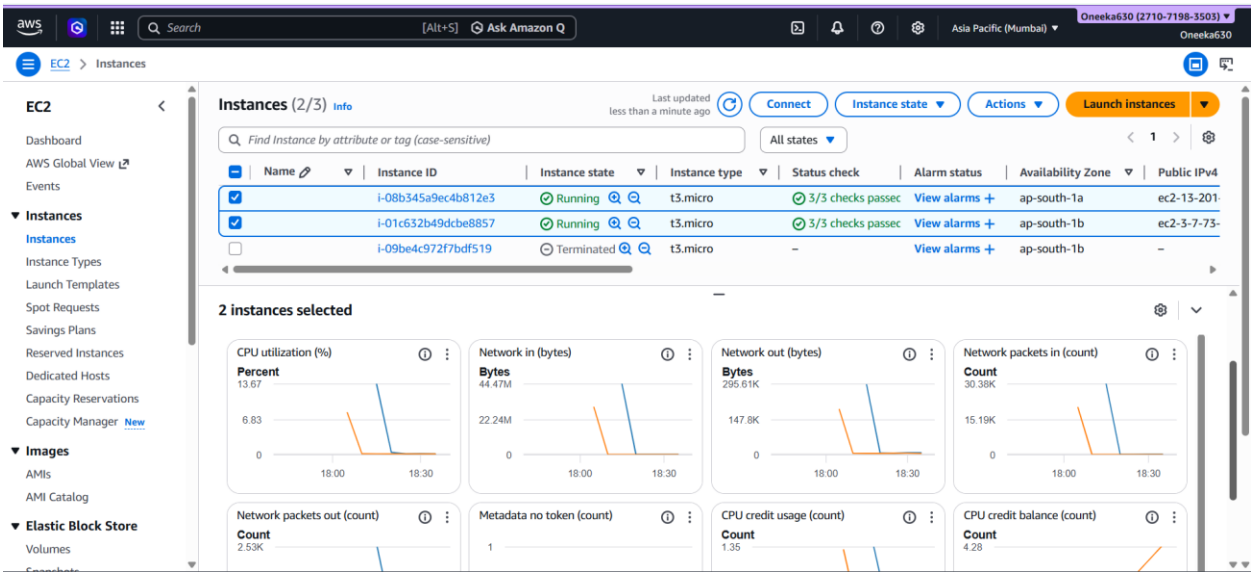
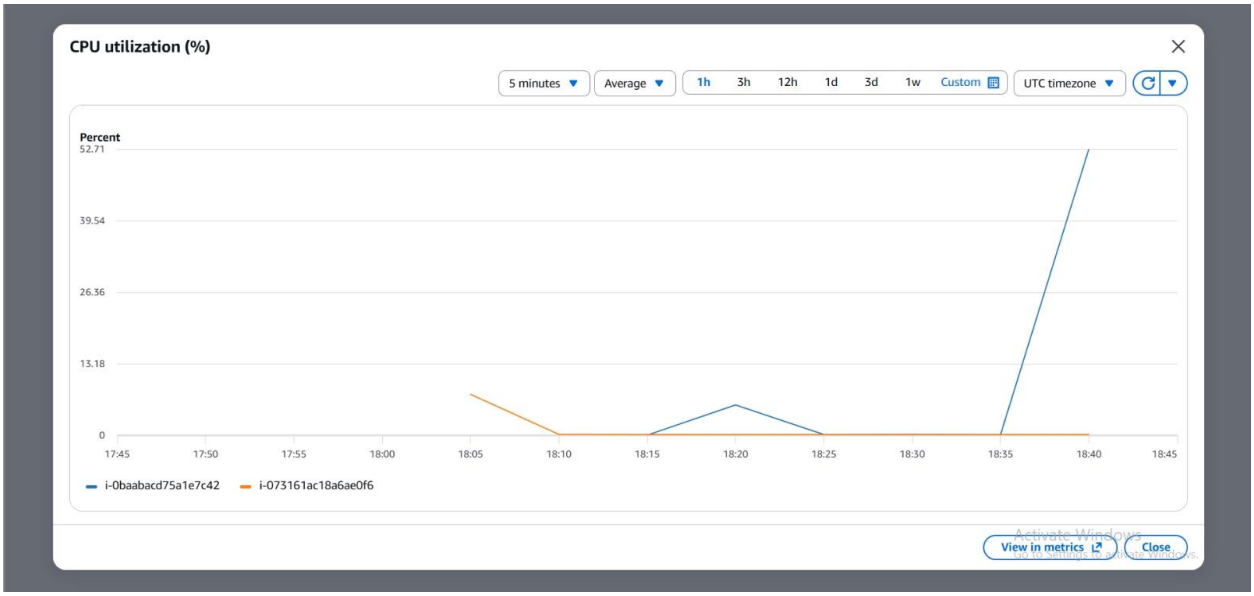
Step 22: Inside the terminal type '**sudo nano oneeka.sh**' to create an infinitely running bash script. In the bash script, add the following code.

```
ubuntu@ip-172-31-10-55:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-10-55:~$ sudo nano oneeka.sh
```

Step 23: To make the script executable, type '**sudo chmod +x oneeka.sh**' ->Run the script '**sh oneeka.sh**'.

```
ubuntu@ip-172-31-10-55:~$ sudo chmod +x oneeka.sh
ubuntu@ip-172-31-10-55:~$ sh oneeka.sh
```

Step 24: An infinite loop starts to run, stressing the CPU of the instance -> Selecting the two instances we can look at the CPU utilisation graph.



Step 25: After a while the instance reaches and surpasses it's 50% CPU utilisation threshold. ->So, to take that load on the instance, the auto scaler spins up another instance to balance the load.